

Informe de avance N° 3

Desarrollo del analizador sintáctico con ANTLR 4

Lenguajes de Programación y Compiladores - UNSE FCEyT - 2026

Lenguaje: DFD Lang

Grupo: Los Lexemas

Integrantes: Gastón Coronel, Jerónimo Gili, Leonardo Noriega y Jorge Taboada.

Docentes: MSc. Ing. Margarita Álvarez y Lic. Eugenia Alejandra Gonzalez.

Fecha: 6 de octubre de 2026

1. Objetivo y alcance

Desarrollar con ANTLR 4 un analizador sintáctico para DFD Lang. El parser recibe los tokens de DFDLexer y comprueba si la secuencia puede derivarse de la gramática; además, permite observar el árbol de análisis e informa errores sintácticos con ubicación. El enunciado solicita el desarrollo y las pruebas de esta etapa (Cátedra de Lenguajes de Programación y Compiladores, 2026b, p. 2).

El alcance incluye programas, niveles, bloques, declaraciones DFD, variables, asignaciones, estructuras de control, funciones y expresiones. La comprobación de tipos, identificadores y conexiones corresponde a la etapa semántica y no se valida en este avance.

2. Fundamento teórico

Una gramática libre de contexto describe las construcciones sintácticas mediante terminales, no terminales y producciones. El analizador sintáctico recibe el flujo de tokens del analizador léxico y determina si puede generarse a partir de la gramática (Cátedra de Lenguajes de Programación y Compiladores, 2026a, diap. 4-5). ANTLR genera el parser y el árbol de análisis desde reglas .g4 (ANTLR, s. f.).

3. Gramática implementada

El símbolo inicial es programa. En la notación resumida, * representa cero o más repeticiones y ? una parte opcional. EOF exige consumir toda la entrada. La estructura de expresiones sigue la versión corregida del Informe de avance 2 (Coronel et al., 2026). Las reglas completas están en src/main/antlr4/org/example/DFDParser.g4.

```
programa -> SISTEMA ID ; DefNivel* FIN_SISTEMA EOF
DefNivel -> NIVEL NUMERO Bloque
Bloque -> LLAVE_ABRE Sentencia* LLAVE_CIERRA
Sentencia -> DeclaracionNodo | DeclaracionFlujo | DeclaracionVar
           | Asignacion ; | Condicional | Iterativa | LlamadaFuncion ;
DeclaracionNodo -> TipoNodo ID = CADENA ;
TipoNodo -> ENTIDAD | PROCESO | ALMACEN
DeclaracionFlujo -> FLUJO ID = CONECTAR ( ID -> ID ) CON_DATO CADENA ;
DeclaracionVar -> Tipo ID = Expresion ;
Tipo -> ENTERO | REAL | BOOLEANO
Asignacion -> ID = Expresion
Condicional -> SI ( Expresion ) Bloque ( SINO Bloque )?
Iterativa -> MIENTRAS ( Expresion ) Bloque
LlamadaFuncion -> IMPRIMIR ( Expresion )
                | VALIDAR_MODELO ( NUMERO )
                | CONTAR_NODOS ( TipoNodo )

Expresion -> Disyuncion
Disyuncion -> Conjuncion ( OR Conjuncion )*
Conjuncion -> Negacion ( AND Negacion )*
Negacion -> NOT Negacion | Relacion
Relacion -> Suma ( OpRel Suma )?
OpRel -> == | != | < | > | <= | >=
Suma -> Producto ( + | - ) Producto )*
Producto -> Unaria ( * | / ) Unaria )*
Unaria -> ( + | - ) Unaria | Primaria
Primaria -> ID | NUMERO | CADENA | TRUE | FALSE
           | LlamadaFuncion | ( Expresion )
```

LLAVE_ABRE y LLAVE_CIERRA representan las llaves literales del bloque; Sentencia* indica cero o más sentencias. Precedencia de menor a mayor: OR, AND, NOT/relación, suma/resta, producto/división, unarios y primarias. Relacion admite como máximo un operador relacional, por lo que 1 < 2 < 3 se rechaza. Los signos numéricos se analizan como operadores unarios separados de NUMERO.

4. Integración con ANTLR

DFDParser.g4 utiliza tokenVocab=DFDLexer para reutilizar los tokens existentes. DFDLexer.g4 incorpora REAL, NOT y CADENA_SIN_CIERRE; AND y OR ya estaban definidos. Maven utiliza ANTLR 4.13.2. No se modificaron fuentes Java del proyecto.

```
mvn -q clean package
./tools/verify_parser.ps1
TestRig: org.example.DFD programa -tree <archivo.dfd>
```

El árbol de 2 + 3 * 4 agrupa el producto antes de la suma. En not x < 10 && true || false, la relación queda dentro de NOT, AND agrupa antes de OR y se conserva la precedencia definida.

5. Pruebas del analizador sintáctico

Las entradas inválidas contienen lexemas reconocibles para que el diagnóstico corresponda a la fase sintáctica.

Entrada	Propósito	Resultado
programa_informe1.dfd	Programa del informe anterior con sistema, nivel, nodos y flujo.	Aceptado; 0 errores sintácticos.
sintaxis_precedencia_valida.dfd	Tipos entero/real/booleano, operaciones unarias y binarias, not/and/or, si/sino, mientras y funciones anidadas.	Aceptado; 0 errores sintácticos.
sintaxis_comparacion_encadenada.dfd	Comparación encadenada $1 < 2 < 3$ con todos los lexemas reconocidos.	Rechazado en línea 3, columna 21; 1 error sintáctico.
sintaxis_punto_y_coma_faltante.dfd	Declaración sin punto y coma antes del cierre del bloque.	Rechazado en línea 4, columna 0; 1 error sintáctico.

Diagnósticos de los casos negativos:

```
line 3:21 mismatched input '<' expecting {'&&', '||', '+', '-', '*', '/', ';'}
line 4:0 missing ';' at '}'
```

El resumen de ejecución se conserva en evidencia/salidas/sintaxis.txt y se reproduce con tools/verify_parser.ps1.

6. Cronograma de la etapa

Actividad	Fecha	Responsable	Estado
Revisión de la gramática corregida y alcance	6 de octubre de 2026	Equipo Los Lexemas	Realizado
Ajuste de tokens léxicos requeridos por el parser	6 de octubre de 2026	Equipo Los Lexemas	Realizado
Desarrollo de DFDParse.g4 con niveles de precedencia	6 de octubre de 2026	Equipo Los Lexemas	Realizado
Pruebas de aceptación y rechazo con ANTLR TestRig	6 de octubre de 2026	Equipo Los Lexemas	Realizado
Documentación y preparación del Informe de avance 3	6 de octubre de 2026	Equipo Los Lexemas	Realizado

7. Conclusión

Las reglas implementadas validan la estructura principal de DFD Lang y expresan la precedencia de operadores en niveles separados. Las pruebas aceptan el programa de referencia y expresiones corregidas, y rechazan comparaciones encadenadas y sentencias incompletas. Las validaciones semánticas quedan para la siguiente etapa.

Referencias

Cátedra de Lenguajes de Programación y Compiladores, UNSE-FCEyT. (2026a). A. *sintáctico* [Diapositivas de cátedra].
 Cátedra de Lenguajes de Programación y Compiladores, UNSE-FCEyT. (2026b). *Enunciado del taller 2026* [Consigna].
 Coronel, G., Gili, J., Noriega, L., & Taboada, J. (2026). *Informe de avance 2 corregido* [Documento de trabajo no publicado].
 ANTLR. (s. f.). *ANTLR 4*. <https://www.antlr.org/>